

Projeto Final Integrador

Título: Proposta de Arquitetura Cloud-Native para Sistemas Escaláveis com Microsserviços, Serverless e Observabilidade

Autor: Gabriel Ribeiro de Camargo Miranda

Data: Junho/2026

Resumo

Este trabalho apresenta uma proposta técnica e científica de arquitetura cloud-native voltada ao desenvolvimento de sistemas escaláveis, seguros e observáveis. A solução integra front-end desacoplado, camadas de API, processamento serverless, microsserviços em contêineres e persistência híbrida, combinando bancos relacionais e NoSQL conforme a natureza do dado e do fluxo transacional. A proposta foi fundamentada em literatura sobre microsserviços, computação serverless, segurança em sistemas distribuídos e boas práticas de engenharia em nuvem. Além disso, a arquitetura foi consolidada com foco em elasticidade, resiliência, comunicação assíncrona e governança operacional, buscando responder à necessidade de modernização de aplicações que ainda operam sob modelos monolíticos. Como resultado, o documento organiza a solução em torno de requisitos não funcionais críticos, conectando problema, fundamentação teórica, arquitetura proposta e serviços em nuvem. Conclui-se que a combinação de padrões arquiteturais maduros com serviços gerenciados em nuvem aumenta a capacidade de evolução do sistema, reduz complexidade operacional e melhora a previsibilidade de desempenho.

Palavras-chave: arquitetura cloud-native; microsserviços; serverless; computação em nuvem; escalabilidade.

1. Introdução

A evolução dos sistemas de informação nas últimas décadas ampliou a necessidade de arquiteturas capazes de suportar crescimento, variabilidade de carga e evolução contínua. Em cenários corporativos e acadêmicos, a pressão por entregas rápidas e por integração com múltiplos serviços tornou insuficientes os modelos rígidos de desenvolvimento baseados em aplicações monolíticas. Nesse contexto, a computação em nuvem e as arquiteturas distribuídas passaram a ocupar papel central na modernização de soluções digitais, especialmente quando

associadas a práticas como automação de implantação, observabilidade e provisionamento elástico [1][2].

O presente projeto final integrador consolida as atividades realizadas ao longo da disciplina e propõe uma arquitetura que combina front-end desacoplado, API Gateway, funções serverless, microsserviços em contêineres, autenticação centralizada e persistência híbrida. A proposta também considera princípios do AWS Well-Architected Framework, com ênfase em confiabilidade, segurança, eficiência de performance e otimização de custos [9]. Essa combinação permite discutir não apenas a construção de uma solução tecnicamente coerente, mas também a relação entre teoria e prática no desenvolvimento de sistemas escaláveis.

2. Problema abordado

O problema tratado neste trabalho é a dificuldade de evoluir sistemas de software que cresceram de forma orgânica e sem direção arquitetural clara. Em muitos casos, a aplicação passa a concentrar regras de negócio, acesso a dados, interface e integração externa em uma única base de código, o que gera dependências excessivas, baixa testabilidade e alto custo de manutenção. Quando essa estrutura é migrada para a nuvem sem revisão arquitetural, os benefícios de escalabilidade e elasticidade são parcialmente perdidos [2][9].

Além disso, a ausência de observabilidade e de fronteiras bem definidas entre componentes dificulta a identificação de falhas e a adequação da solução a picos de uso. Estudos sobre microsserviços mostram que a decomposição do sistema em serviços independentes melhora a evolução técnica, mas também introduz complexidade adicional de comunicação, segurança e monitoramento [2][8]. A proposta deste projeto responde a esse cenário ao estruturar uma arquitetura que distribui responsabilidades, reduz acoplamento e explora serviços gerenciados em nuvem para minimizar esforço operacional.

3. Fundamentação teórica

A literatura sobre microsserviços aponta que a separação de uma aplicação em serviços pequenos e focados em uma única capacidade de negócio facilita o desenvolvimento independente e o escalonamento seletivo [1][2]. Newman destaca que a divisão deve ser orientada por limites de domínio, e não por conveniência técnica, para evitar a criação de um monólito distribuído. Cerny et al. reforçam que os ganhos de escalabilidade e deploy independente precisam ser equilibrados com maior complexidade operacional, especialmente

em testes, observabilidade e comunicação entre serviços [2].

No campo serverless, os levantamentos de Shafiei et al., John e Gupta, Li et al. e Wen et al. demonstram que esse paradigma é especialmente útil em aplicações event-driven, workloads variáveis e tarefas de curta duração, pois reduz a necessidade de gerenciamento de infraestrutura e permite pagamento por uso [3][4][5][6]. Brooker et al. mostram ainda que a capacidade de carregar contêineres sob demanda ampliou o espaço de uso do AWS Lambda, aproximando o modelo serverless de aplicações com dependências maiores, sem abrir mão de escalabilidade automática [7].

A segurança em ambientes distribuídos também é um ponto central. Indrasiri e Siriwardena defendem que microsserviços exigem autenticação consistente, autorização centralizada, comunicação protegida e isolamento de rede mais rigoroso que o de aplicações monolíticas [8]. Em arquiteturas cloud-native, a adoção de gateways, tokens JWT e serviços de identidade reduz a dispersão de mecanismos de acesso e fortalece o controle de identidade. Por fim, o AWS Well-Architected Framework orienta decisões de projeto ao enfatizar confiabilidade, segurança, excelência operacional, eficiência de performance e otimização de custos [9].

4. Arquitetura proposta

A arquitetura proposta organiza o sistema em cinco camadas principais: apresentação, integração, processamento, persistência e observabilidade. Na camada de apresentação, o front-end é implementado como aplicação web desacoplada, servida por CDN e armazenamento estático, preferencialmente com CloudFront e S3, o que reduz latência e facilita atualização independente da interface [15].

Na camada de integração, o API Gateway centraliza o roteamento das requisições e aplica políticas de acesso, limite de requisições e versionamento de endpoints [10]. Essa camada atua como ponto único de entrada, reduzindo acoplamento entre cliente e serviços internos. A autenticação é concentrada em um provedor de identidade como o Amazon Cognito, o que simplifica login, emissão de tokens e controle de permissões [12].

Na camada de processamento, a proposta combina AWS Lambda para fluxos síncronos e orientados a evento, e microsserviços em containers para tarefas mais longas ou com dependências específicas [11]. Essa abordagem híbrida é justificável porque nem toda rotina se adapta ao mesmo modelo de execução. Funções curtas e reativas se beneficiam de serverless,

enquanto serviços de domínio com maior complexidade podem ser isolados em contêineres. Essa distribuição melhora a adaptabilidade do sistema e evita a adoção forçada de um único paradigma [3][7].

Na persistência, o modelo híbrido usa Amazon RDS para dados relacionais e DynamoDB para dados semiestruturados, sessões e registros de alto volume [13][14]. A escolha é orientada pela natureza do dado: transações críticas e relacionamentos complexos permanecem no banco relacional, enquanto eventos e estruturas flexíveis ganham desempenho em NoSQL. Para suporte a integrações e desacoplamento entre módulos, o uso de filas ou tópicos em serviços como SQS e SNS é indicado para comunicação assíncrona.

5. Serviços em nuvem

Os serviços em nuvem são fundamentais para transformar a arquitetura proposta em uma solução operacionalmente viável. O CloudFront distribui conteúdo estático com cache em borda, reduzindo a carga sobre a origem. O S3 armazena a aplicação front-end e artefatos estáticos. O API Gateway oferece gerenciamento de tráfego, validação de chamadas e integração com mecanismos de autenticação [10][15].

O AWS Lambda executa tarefas sob demanda e reduz o esforço com provisionamento e manutenção de servidores [11]. O uso de Lambda é adequado para endpoints de baixa latência, rotinas de integração e processamento em lote de pequena duração [3][4]. Em cenários com maior controle sobre runtime e dependências, o ECS/Fargate permite empacotar serviços em containers e manter autonomia de implantação. Para dados, o RDS garante consistência transacional, enquanto o DynamoDB oferece baixa latência e elasticidade para cargas imprevisíveis [13][14].

Do ponto de vista operacional, CloudWatch e X-Ray sustentam logs, métricas e tracing distribuído. Isso permite acompanhar latência, taxa de erro e gargalos entre componentes, o que é essencial em sistemas distribuídos [9]. Em conjunto, os serviços gerenciados reduzem complexidade de infraestrutura e permitem concentrar esforço na lógica de negócio.

6. Segurança e escalabilidade

A segurança da solução é estruturada por camadas. O acesso externo passa por CloudFront e API Gateway, reduzindo exposição direta de recursos internos. A autenticação centralizada via Cognito ou JWT permite padronizar identidade e autorização. Em rede, a segmentação em sub-

redes privadas para persistência e processamento diminui a superfície de ataque [8][9].

A escalabilidade é tratada de forma horizontal e elástica. Funções Lambda escalam automaticamente com a demanda, enquanto contêineres podem ser replicados conforme métricas de consumo. O banco relacional pode utilizar réplicas de leitura e o DynamoDB pode operar sob demanda em cenários de tráfego variável. A comunicação assíncrona por filas e tópicos desacopla produtor e consumidor, reduzindo impacto de falhas e suportando picos sem bloqueio da aplicação principal [3][6][7].

Além disso, a arquitetura considera resiliência por meio de redundância em zonas de disponibilidade, retry com backoff exponencial e observabilidade contínua. Essas práticas aumentam a capacidade do sistema de continuar operando mesmo diante de falhas parciais, o que é coerente com recomendações de arquiteturas orientadas a serviços e com o enfoque do AWS Well-Architected Framework [9].

7. Considerações finais

Este trabalho consolidou uma proposta arquitetural que articula teoria e prática no contexto da iniciação científica em engenharia de software. A solução apresentada demonstra que a adoção de microsserviços, serverless e serviços gerenciados em nuvem não deve ser vista como tendência isolada, mas como resposta técnica a requisitos concretos de escalabilidade, segurança e manutenção.

Conclui-se que a arquitetura proposta oferece um caminho consistente para modernização de sistemas, especialmente quando há necessidade de separar responsabilidades, apoiar crescimento de usuários e reduzir custo operacional. Como continuidade natural, recomenda-se detalhar um protótipo funcional, validar a solução por meio de testes de carga e complementar a documentação com métricas de desempenho e segurança.

8. Referências

- [1] NEWMAN, S. *Building Microservices: Designing Fine-Grained Systems*. 2. ed. Sebastopol: O'Reilly Media, 2021.
- [2] CERNY, T.; BUSHEY, R.; TCHAKOUNTÉ, F. et al. On Microservices Architecture: A Systematic Mapping Study. *The Journal of Systems and Software*, v. 142, p. 72-85, 2018. DOI: 10.1016/j.jss.2018.04.058.

- [3] SHAFIEI, H.; KHONSARI, A.; MOUSAVI, P. Serverless Computing: A Survey of Opportunities, Challenges and Applications. *arXiv*, 2019. Disponível em: <https://arxiv.org/abs/1911.01296>. Acesso em: 09 jun. 2026.
- [4] JOHN, J.; GUPTA, S. A Survey on Serverless Computing. *arXiv*, 2021. Disponível em: <https://arxiv.org/abs/2106.11773>. Acesso em: 09 jun. 2026.
- [5] LI, Z. et al. The Serverless Computing Survey: A Technical Primer for Design Architecture. *arXiv*, 2021. Disponível em: <https://arxiv.org/abs/2112.12921>. Acesso em: 09 jun. 2026.
- [6] WEN, J. et al. Rise of the Planet of Serverless Computing: A Systematic Review. *arXiv*, 2022. Disponível em: <https://arxiv.org/abs/2206.12275>. Acesso em: 09 jun. 2026.
- [7] BROOKER, M. et al. On-demand Container Loading in AWS Lambda. *arXiv*, 2023. Disponível em: <https://arxiv.org/abs/2305.13162>. Acesso em: 09 jun. 2026.
- [8] INDRASIRI, K.; SIRIWARDENA, P. *Microservices Security in Action*. Shelter Island: Manning Publications, 2020.
- [9] AWS. AWS Well-Architected Framework. Disponível em: <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>. Acesso em: 09 jun. 2026.
- [10] AWS. Amazon API Gateway Developer Guide. Disponível em: <https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html>. Acesso em: 09 jun. 2026.
- [11] AWS. AWS Lambda Developer Guide. Disponível em: <https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>. Acesso em: 09 jun. 2026.
- [12] AWS. Amazon Cognito Developer Guide. Disponível em: <https://docs.aws.amazon.com/cognito/latest/developerguide/what-is-amazon-cognito.html>. Acesso em: 09 jun. 2026.
- [13] AWS. Amazon RDS User Guide. Disponível em: <https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html>. Acesso em: 09 jun. 2026.
- [14] AWS. Amazon DynamoDB Developer Guide. Disponível em:

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html>.

Acesso em: 09 jun. 2026.

[15] AWS. Amazon CloudFront Developer Guide. Disponível em:

<https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Introduction.html>.

Acesso em: 09 jun. 2026.